

Agenda:

1. How to approach schema design
2. MVC Architecture
3. Design factors
4. Class Diagram

Schema design

1. For each entity created we create a table.
2. For any primitive properties, we simply add a column to the table.
(Int, float, boolean, string, double)

For eg : class Movie {

String name;
int runtime;
String genre;

Actor actor;

}

} primitive properties.

→ reference / non-primitive property.



movie

id	name	runtime	genre
----	------	---------	-------

3. For non-primitive / reference properties we have to follow the rules of cardinality.

Rules of Cardinality:

One-to-one (1:1)

Husband	wife	Class Husband {	Class Wife {
1	:	String name;	String name;
1	:	int age;	int age;
1	:	wife wife;	husband husband;
1	:	}	}

Husband	wife
id name age wife-id	id name age

- We will store any one entity's id on the other entity table and vice versa.

One-to-many (1:M) [vice versa]

Student	Batch	Class Student {	Class Batch {
1	:	String name;	String name;
M	:	int age;	int strength;
M	:	Batch batch;	List<Student> students;
M	:	}	}

Student

id	name	age	batch_id
----	------	-----	----------

Batch

id	name	strength	list of id-id
----	------	----------	--------------------------

- Store 'one's entity ID on the 'many's entity table.

Many-to-Many (M:M)

Movie : Actor

1 : M

M : 1

M : M

Class Movie {

String name;

int runtime;

List<Actor> actors;

}

Class Actor {

String name;

int age;

List<Movie> movies;

}

Movie

id	name	runtime
----	------	---------

Actor

id	name	age
----	------	-----

Movie Actor Map

id	movie_id	actor_id
----	----------	----------

id	movie_id	actor_id
id1	sholay-id	amitabh-id
id2	sholay-id	dharma-id
id3	baghtan-id	amitabh-id
id4	don-id	amitabh-id

MVC Architecture:

Seating area → waiter → Kitchen (chefs) → pantry → freezer

view → controller → service (business) → repository → models.
↓
~~JSPs~~
APIs.
- getBatchByStdIdC(),
- "Select * from std ..."

controller → responsible for managing user interaction with backend [input/output formatting]

service → responsible for business related logic.

repository → common code to fetch data from models(DB)

models → Entity Classes [ORM → Object Relation mapping]

class {
 ||
 ||
 ||
 ||
}

← ORM →

~		~		~		~
---	--	---	--	---	--	---

Break time : 10:30 pm
→

Design Tic Tac Toe

Step 0 : Getting an overview

- 1) Entity or full application ? → full application
- 2) Do we persist data → Not persist.
- 3) Input from user → command line.

Step 1 : Requirement Gathering:

↳ visualisation → User Journey
Physical structure

- 1) Users should be able to start a game.
- 2) Users can select size of board : $N \times N$

, ## ,

- 3) No. of players can be configured : $2 \leq N-1$
- 4) Will there be bot players ? Yes (1 bot per game)
- 5) Bot player will have diff difficulty levels.
↳ HARD, EASY, MEDIUM
- 6) Player can customise their name &

symbol . → Check if symbols are unique .

7) Who will start the game ?

↳ Random or customise order of players .

8) Undo moves ? → Yes .

9) Will the hints ? → NO

10) Pause the game → NO

11) Will there be timer ? NO

12) Player move history ? NO .

13) Check Winner logic ?

↳ consecutive Row ☒

↳ consecutive Column ☒

↳ consecutive diagonal ☐

↳ all corners . ☐

14) Let users select which logic or combination of logic they want to play .

Advanced feature :

1. Leaderboard

2. Tournaments .

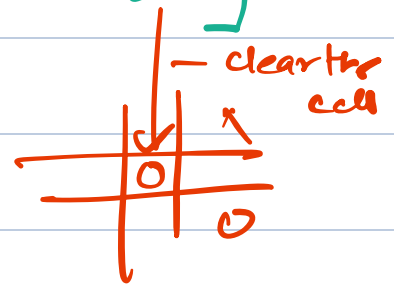
How to implement the Undo feature?

1) Kuch Kuch nota hai : (Some things happen)

- Store moves in a list.
- Remove the last move from list.
- Clear the location of the cell from board.

$$\text{list} \langle \text{move} \rangle = \left[\left\{ \begin{matrix} x, \\ y, \\ \text{symbol} \end{matrix} \right\}, \left\{ \begin{matrix} 0, \\ 1, \\ x \end{matrix} \right\} \dots \left\{ \begin{matrix} 2, \\ 2, \\ 0 \end{matrix} \right\} \right]$$

$$\begin{array}{c|c|c} 0 & 1 & 2 \\ \hline 0 & & \\ \hline 1 & & x \\ \hline 2 & & \end{array} \rightarrow \left\{ \begin{matrix} 1, \\ 2, \\ x \end{matrix} \right\}$$



2) Om Shanti Om (Rebirth)

	0	1	2	3	4
0	A	D	D	D	D
1	O ₁	O ₂	O ₃		O ₅
2			O ₃	O ₅	
3	O ₁	O ₂		O ₃	
4	A	A	A	O	A

→

$$\left[\left\{ \begin{matrix} x, \\ y, \\ \text{symbol} \end{matrix} \right\} \dots \left\{ \begin{matrix} 3, \\ 2, \\ 0 \end{matrix} \right\} \right]$$

- store moves in a list
- we will remove last move from list.

→ Reset the whole board to 0 start of game

$$\left[\begin{matrix} \{2, 2, 0_3\}, \\ \{3, 2, 0_4\}, \\ \{3, 2, 0_5\}, \\ \{2, 2, 0_3\} \end{matrix} \right]$$

→ internally play every move from the list (except

last one as it is deleted)

Time Complexity : $O(n)$
 $\hookrightarrow$ moves.

	0	1	2	3	4
0	A	D	D	D	D
1	O ₁	O ₂			O ₃
2			O ₃	O ₅	
3	O ₁	O ₂		O ₃	
4	A	A	A	O	A

3) Doeramon's anywhere door ? Time Comp : $O(1)$

i) Store snapshot of board as list.

	0	1	2	3	4
0	A	D	D	D	D
1	O ₁	O ₂			O ₃
2			O ₃	O ₅	
3	O ₁	O ₂		O ₃	
4	A	A	A	O	A

,

	0	1	2	3	4
0	A	D	D	D	D
1	O ₁	O ₂			O ₃
2			O ₃	O ₅	
3	O ₁	O ₂		O ₃	
4	A	A	A	O	A

,

	0	1	2	3	4
0	A	D	D	D	D
1	O ₁	O ₂			O ₃
2			O ₃	O ₅	
3	O ₁	O ₂		O ₃	
4	A	A	A	O	A

Space Complexity : $O(n)$
 $\hookrightarrow$ n is moves.